Computers & Security 158 (2025) 104644

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Cordon: Enhancing security through kernel-level control in containerized computing environments

Qiqing Deng [a,b,c], Zhen Xu [b,c,*], Qihui Zhou [b], Yan Zhang [b,c]

[a] Beijing Police College, Beijing, China
[b] Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[c] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

ARTICLE INFO

Keywords:
Container security
Security policy
Kernel-level control
eBPF

ABSTRACT

Containers have become a foundational technology across a variety of computing environments, enabling an era of agility, efficiency, and scalability due to their inherent advantages. Simultaneously, containers confront escalating security threats, with vulnerabilities being exploited to compromise host machines and broaden attack impacts. Existing security mechanisms predominantly rely on host-based mandatory access control, which contradicts the autonomy and flexibility requirements of dynamic and scalable containerized computing environments. This paper introduces Cordon, a novel framework aimed at providing autonomous and flexible control management within the context of containerized computing, effectively addressing the limitations of existing security mechanisms. Cordon is designed to counter common attack vectors in containerized environments by implementing file access control, capability management, and system call interception, thereby enabling comprehensive container-aware security enforcement at the kernel level. Furthermore, Cordon supports multi-container management, enabling the application of security policies across various dimensions of container resources, a feature that allows for the batch security management of containers of the same type, such as multiple container instances deployed under the same Kubernetes deployment. We develop a prototype implementation of Cordon and evaluate its effectiveness, generality, and performance overhead. Our evaluation demonstrates that Cordon effectively blocks various container attacks while maintaining acceptable overhead.

1. Introduction

Containers have become a foundational technology across diverse computing environments due to their flexibility, efficiency, and scalability (Pérez et al., 2018; Kaur et al., 2017). Simultaneously, the widespread adoption of containers attracts increased attention from attackers (Tak et al., 2017; Yang et al., 2021; Sultan et al., 2019). Within the shared kernel architecture, attackers employ various strategies to compromise the isolation between container instances and the host, thereby jeopardizing the security of the host and other containerized applications. For instance, the synergistic power attack discussed in Gao et al. (2018) exploits information leakage vulnerabilities in containerized environments, while the container escape attacks in sandeepseeram (2021), Lee et al. (2019) leverage special capabilities to perform arbitrary file read/write operations, and the latest eBPF-based cross-container attack that exploits offensive extended Berkeley Packet Filter (eBPF) features in He et al. (2023). Implementing access control for containers is an effective countermeasure to enhance security, as it can prevent compromised containers from performing unauthorized actions, thereby minimizing the attack surface.

Access control techniques have been extensively studied in the context of traditional virtual machines or hosts and are now increasingly applied to container security (Schreuders et al., 2013; Brimhall et al., 2023; M.P. et al., 2016). Current research on security access control for containers can be categorized into two main approaches: one applies well-known Linux Security Modules (LSM) such as AppArmor and SELinux to container environments, as demonstrated by the works of Docker-sec (Loukidis-Andreou et al., 2018a) and Lic-Sec (Zhu and Gehrmann, 2021a), simplifying the creation of access control policies for container instances; the other explores the utilization of eBPF programs for intercepting kernel functions to implement access control, as seen in studies like Bouheki (mrtc0 et al., 2022), BPFBox (Findlay et al., 2020), and Bpfcontain (Findlay et al., 2021), which enable more convenient monitoring of various system behaviors. Despite the effectiveness of these methods in mitigating certain container security risks, existing container-oriented access control solutions still face challenges in containerized computing environments that heavily depend on dynamism and scalability.

* Correspondence to: Institute of Information Engineering, Chinese Academy of Sciences, Beijing, 100085, China.
E-mail addresses: dengqiqing@bjpc.edu.cn (Q. Deng), xuzhen@iie.ac.cn (Z. Xu), zhouqihui@iie.ac.cn (Q. Zhou), zhangyan@iie.ac.cn (Y. Zhang).

https://doi.org/10.1016/j.cose.2025.104644
Received 1 November 2024; Received in revised form 8 June 2025; Accepted 21 August 2025
Available online 30 August 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Firstly, containers hosting different applications usually have distinct security requirements (Flauzac et al., 2020; Souppaya et al., 2017), but existing access control solutions like Bouheki and BPFBox lack container-level security considerations and do not support per-container policies. Secondly, containerized computing environments enhance resource utilization through scalability (Aruna and Pradeep, 2020; Ahmad et al., 2022), which demands flexible security configurations, such as applying the same policy to instances of similar applications. However, most current access control research overlooks this crucial need for flexible policy extension in containerized computing environments. Therefore, security mechanisms designed for containerized environments must align with their core advantages, ensuring robust protection without compromising the flexibility and scalability of containerized systems. This necessitates innovative security approaches that provide autonomous control and flexible configuration while maintaining non-intrusiveness to preserve the operational efficiency offered by containers.

To address the complex challenges of autonomous control and flexible configuration in containerized computing environments, we introduce Cordon, a fine-grained access control solution designed to enhance security within containerized computing environments. Initially, Cordon leverages namespaces strategically to differentiate container instances, thus embodying the concept of autonomous container security control and enabling targeted security implementations for individual containers. Furthermore, Cordon incorporates a multifaceted policy enforcement setup, allowing for flexible and detailed monitoring and management of behaviors across various resource dimensions, including processes, instances, deployments, and nodes, thereby ensuring scalable and secure batch control of container applications. Cordon design has been successfully implemented and tested with various container engines. Evaluation shows that Cordon effectively defends against various types of attacks while maintaining a low performance overhead (<4% on our use cases).

Contributions. In summary, we make the following contributions:

- We conduct an in-depth investigation into the demand for autonomous and flexible security controls in containerized computing environments, presenting an extensive analysis and innovative strategy designs.
- We unveil Cordon, a groundbreaking fine-grained access control solution based on eBPF and LSM technologies, specifically addressing the critical needs for autonomy and adaptability in container security.
- Through empirical assessments, we demonstrate Cordon's effectiveness in mitigating container-specific attack vectors, showcasing its seamless integration with mainstream container engines and minimal performance overhead. This positions Cordon as an essential advancement for security in containerized computing.

The rest of this paper proceeds as follows. Next section discusses the background. In Section 3, we articulate the motivation driving our work, highlighting the need for autonomous and scalable security solutions. Section 4 defines the threat model, outlining the specific risks addressed by our approach. The design of our proposed framework, Cordon, is detailed in Section 5, followed by an in-depth exploration of its key technical components in Sections 6 and 7. Section 8 evaluates Cordon's effectiveness, generality, and performance through comprehensive experiments. Section 9 offers a discussion of our findings and identifies avenues for future research, while Section 10 concludes the paper with a summary of our contributions.

2. Background

This section establishes the foundational knowledge essential for understanding our research on container security controls. We begin by exploring Linux Security Modules (LSM), a critical framework for enforcing security policies at the kernel level, which forms the basis for many access control mechanisms in containerized environments. Following this, we introduce the extended Berkeley Packet Filter (eBPF), a versatile technology that enables dynamic, real-time monitoring and intervention within the system. Together, these technologies provide the necessary context for appreciating the innovations presented in our work, as they address both the enforcement and adaptability challenges inherent in securing containerized systems.

2.1. Linux security modules

Linux Security Modules (LSM) form a critical security framework within the Linux kernel, integrating configurable hooks at key security checkpoints to enable users to enforce tailored security policies via different modules (Wright et al., 2002). This mechanism strengthens system resilience, particularly against malicious threats in cloud environments. Initially developed to offer a versatile and extensible security architecture, LSM has gained traction through implementations like SELinux and AppArmor, which are widely deployed across various use cases. However, these traditional frameworks were not designed with containerized environments in mind, revealing limitations in addressing container-specific security needs. For example, their reliance on global configurations conflicts with the demand for autonomous, fine-grained control in individual container instances, rendering them less effective in multi-tenant container setups. This mismatch highlights the urgent need for innovative solutions to adapt LSM capabilities to the dynamic requirements of modern container security.

2.2. Extended Berkeley Packet Filter

Extended Berkeley Packet Filter (eBPF) is a versatile technology that allows sandboxed programs to run within the Linux kernel, enabling dynamic functionality extensions without altering kernel source code or requiring system reboots (Rice, 2023). It simplifies development through a streamlined programming model, contrasting with the complexity of traditional kernel modules. A key feature is its built-in verifier, which ensures safety by checking for issues like infinite loops or memory violations before execution, maintaining kernel stability. This design supports secure program loading and unloading, facilitating real-time policy adjustments. By hooking into kernel subsystems, eBPF captures detailed system events – such as process execution, network traffic, and file operations – making it invaluable in containerized environments where shared kernels demand robust monitoring. Its efficiency and adaptability enhance security detection and system observability, underscoring its growing role in modern computing.

3. Motivation

In this section, we explore the urgent need for autonomous security controls and flexible policy configuration in containerized computing environments. We proceed by analyzing the primary challenges that obstruct the implementation of these vital capabilities. To illustrate their importance, we then introduce a motivating example that highlights the critical role of enabling independent security management and adaptable control within container systems.

3.1. Security requirements in containerized computing environments

Containerized computing environments have revolutionized cloud-native architectures through lightweight virtualization, enabling efficient resource sharing across isolated instances via kernel namespaces and cgroups (Watada et al., 2019). This paradigm supports dynamic orchestration platforms like Kubernetes, which automate container scaling to meet fluctuating workloads. However, the shared-kernel architecture that drives this efficiency simultaneously introduces intrinsic

security tensions. The absence of strict hardware-level isolation necessitates security mechanisms to compensate for weaker trust boundaries between containers and hosts, particularly in multi-tenant deployments where heterogeneous workloads coexist (Sun et al., 2018).

Three critical security requirements emerge from these environments:

(a) **Instance-Specific Policy Customization.** Containers often host applications with divergent security profiles—e.g., high-through put data processors requiring extensive I/O system calls versus financial services demanding strict filesystem access controls. Global security frameworks like AppArmor, designed for host-wide enforcement, fail to address such per-container granularity. For instance, a database container may require 'CAP_DAC_READ_SEARCH' capability auditing to prevent privilege escalation, while a web server container needs network policy enforcement. Existing kernel security modules lack container context awareness, forcing administrators to choose between over-permissive policies (risking container escapes) or overly restrictive configurations (disrupting legitimate operations).

(b) **Batch Security Management.** Orchestrators dynamically scale container replicas (e.g., Kubernetes ReplicaSets), yet security frameworks remain siloed to individual instances. A payment processing microservice with 50 replicas requires uniform security policies across all instances, but current tools like SELinux demand manual per-container configuration. This gap complicates compliance auditing and increases misconfiguration risks—a challenge exacerbated during auto-scaling events where new containers inherit outdated or incompatible policies.

(c) **Dynamic Threat Adaptation.** Containerized workloads frequently update dependencies and runtime configurations, altering their attack surfaces. A container initially restricted from 'mount' syscalls may later require temporary access for debugging, necessitating policy adjustments without service interruption. Traditional Mandatory Access Control (MAC) systems enforce static rules, creating operational friction in DevOps pipelines.

The core challenge lies in reconciling these requirements with the constraints of kernel-level enforcement. While Linux capabilities and namespace isolation provide foundational security guarantees, their coarse-grained implementation fails to mitigate advanced threats such as container escape attacks (Li et al., 2023). We observe that these breaches exploit the discrepancies between container-aware policies and kernel-enforced permissions. For instance, privileged containers often inadvertently inherit excessive capabilities (e.g., 'CAP_DAC_READ_SEARCH'), enabling attackers to bypass namespace boundaries and execute arbitrary file reads on the host.

These limitations underscore the need for a dual approach: (1) fine-grained container context capture (e.g., cgroup memberships, namespace identifiers) to enable instance-specific policy binding, and (2) programmable security primitives that scale with orchestration dynamics. Emerging solutions must bridge the semantic gap between kernel-level event monitoring (e.g., eBPF tracing) and container-specific threat models—ensuring policies evolve alongside ephemeral workloads without compromising performance or operational agility.

3.2. A motivating example

As explained in the previous subsection, while kernel security modules like SELinux and AppArmor enforce mandatory access controls, their host-wide policy scope proves inadequate for multi-tenant scenarios. As illustrated in Fig. 1 (right), these frameworks treat all containers as homogeneous entities, applying uniform rules regardless of workload criticality—a database container requiring strict filesystem auditing receives identical treatment to a stateless web server. This "one-size-fits-all" approach creates two systemic risks: over-privileged

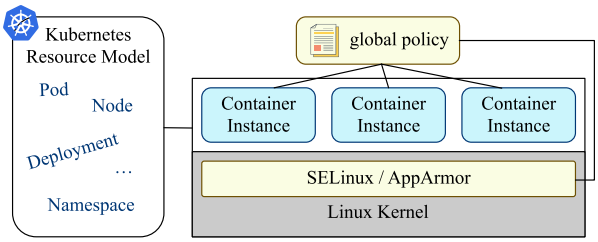


Fig. 1. Landscape of existing kernel security frameworks within containerized computing scenarios. On the left, it outlines the various resource models inherent to Kubernetes, the leading container orchestration platform, while the right illustrates the global nature of current kernel security frameworks.

containers vulnerable to capability abuse (e.g., 'CAP_SYS_PTRACE' enabling container escapes) and under-permissioned containers triggering false-positive denials during legitimate operations.

The challenge intensifies with orchestration-driven scalability. Kubernetes deployments (Fig. 1, left) dynamically spawn identical replicas (e.g., 50 web server pods) to handle traffic spikes. Existing security tools lack resource-model awareness, forcing administrators to manually configure policies per container—a process error-prone at scale. Consider a web tier compromised via a privileged sidecar container: attackers exploit 'CAP_DAC_READ_SEARCH' to access host-mounted '/etc/shadow'. Traditional LSMs would either globally block the capability (disrupting legitimate use) or allow it universally (enabling cross-replica breaches). Neither approach addresses the need for granular, workload-aware controls.

Our solution leverages eBPF's programmability to bridge this gap. By intercepting LSM hooks and correlating kernel events with container identities (docker IDs, pod labels), we enforce context-sensitive policies without kernel modifications. For instance, administrators can define a rule blocking 'CAP_DAC_READ_SEARCH' for all pods labeled 'tier: web'—automatically applied to new replicas during scaling events. This approach reduced configuration errors, demonstrating how kernel-aware container context transforms security from a static barrier into an adaptive, orchestration-native layer.

4. Threat model and assumptions

In this section, we present the threat model and assumptions underpinning our research on kernel-level security control for containerized computing environments. We begin by outlining the key assumptions that frame our approach, followed by a data flow diagram (DFD) that maps the container creation and runtime process. Next, we identify critical vulnerabilities inherent to containerized environments, define the potential consequences of threats, and analyze two primary threat categories: information disclosure and elevation of privilege. Finally, we specify the types of threats our defense mechanism is designed to mitigate, emphasizing kernel-level security challenges.

4.1. Assumptions

As noted in the motivation, existing solutions fall short in delivering autonomous security control and batch management for containerized computing environments. To address this, we propose a kernel-level security control scheme that reframes distributed security management as a host-based problem. We assume this scheme can be deployed as an agent across multiple worker nodes, enabling batch autonomous security control of containers within a cluster. Accordingly, our study focuses the attack scope on a single worker node, targeting threats faced by containers on an individual host. Our approach rests on the following assumptions:

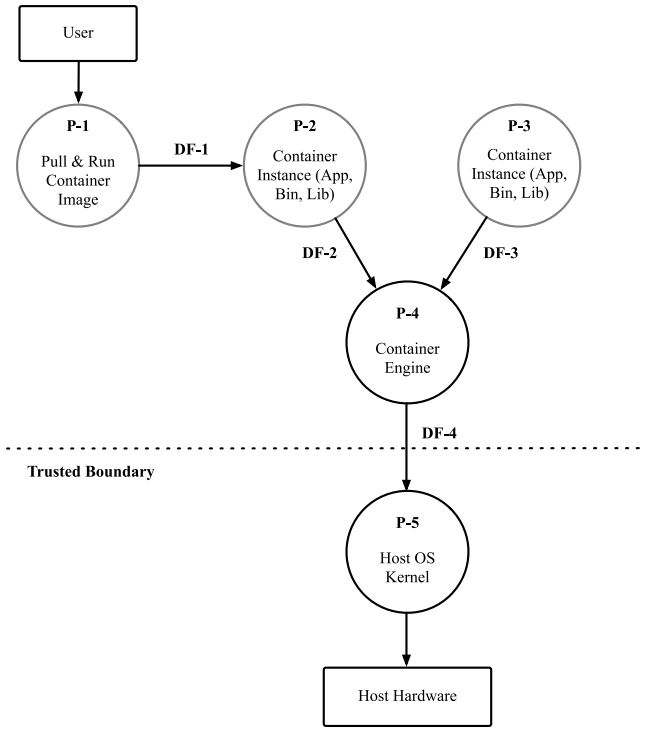


Fig. 2. Data flow diagram of the container system.

1. **Attacker Capabilities:** Attackers can exploit compromised containers to breach isolation mechanisms, gaining unauthorized access to the host and potentially jeopardizing the stability and security of the entire cloud platform. In containerized environments, where multiple instances often run on a single worker node, containers are more vulnerable to unauthorized access than traditional host-based applications. The shared kernel and imperfect isolation mechanisms – evidenced by attacks like those in Gao et al. (2018) and Li et al. (2023) – heighten the risk of privilege escalation, making containerized environments particularly susceptible.
2. **Trust Boundaries:** Consistent with prior work on container mandatory access control (Findlay et al., 2021; Bélair et al., 2021; Songi Gwak, 2023), we assume the operating system, interception mechanisms, and audit logs are trustworthy. If the operating system is untrustworthy, our scheme cannot operate securely and is prone to tampering; if interception mechanisms are unreliable, they cannot enforce security controls; and if audit logs are compromised, security analysts cannot rely on them to refine policies or guide defenses. These trust boundaries anchor our focus on vulnerabilities and attack paths arising from compromised container, guiding the design of our permission control scheme.

4.2. Threat modeling using the STRIDE framework

We adopt the STRIDE framework for threat modeling due to its maturity and widespread adoption since 1999. Its data flow-based approach is well-suited to container environments, enabling threat analysis of container components (Wong et al., 2023). The primary aim of this exercise is to delineate our research scope and clarify the defense objectives of our study.

The first step in STRIDE is constructing a data flow diagram (DFD) of the system. For containerized environments, a common use case involves a user pulling a container image and deploying an instance on a worker node. Fig. 2 illustrates the DFD for this process, depicting the container creation and runtime workflow and its key components: A user (external entity) pulls a container image and runs (P-1) a container instance on the host; the host includes five functional components, two of which are container instances (P-2 and P-3); the container instance is an encapsulated, controlled environment housing the application, libraries, and binaries; the container engine (P-4) initiates the container and communicates with the host OS kernel via system calls to manage isolation, capability restrictions, and security settings; and the host OS kernel (P-5) governs hardware and oversees functions like memory, file systems, networking, and process management.

Next, we identified several vulnerabilities in the DFD diagram (Fig. 2) using STRIDE. Each of these vulnerabilities has the potential to cause severe threat consequences. Here, we focus on those vulnerabilities that are most relevant to our research: malicious container images (V1) allow attackers to embed malware by tampering with source code or repositories; software vulnerabilities within containers (V2) arise from flawed applications or libraries, enabling exploitation – particularly when service ports face insecure networks – and risking control over the container and host due to the shared kernel; incomplete isolation mechanisms (V3) stem from imperfect Linux kernel features, such as file isolation flaws that permit access to sensitive host data; and excessive privileges (V4) grant containers unnecessary permissions, exploitable via tools like eBPF to breach isolation boundaries. If exploited, these weaknesses can lead to dire outcomes: unauthorized disclosure (TC-1) compromises sensitive data, deception (TC-2) misleads users with falsified information, disruption (TC-3) interrupts operations, and usurpation (TC-4) hands attackers system control. Recognizing these vulnerabilities and their cascading effects is fundamental to crafting effective security measures for protecting containerized infrastructures.

4.3. Threat analysis

Building on the assumptions in Section 4.1, we identify two primary threat categories in containerized environments: information disclosure and elevation of privilege.

In containerized computing environments, **information disclosure** represents a critical threat to data confidentiality, emerging through two primary mechanisms. Attackers can exploit sensitive parameters in container runtime commands, such as "–privileged" or "–volume", to access host resources illicitly. The "–privileged" flag, for instance, grants root-level access to the host, while "–volume" exposes host file systems, facilitating data exfiltration and process manipulation. Additionally, inter-container information leakage arises when containers sharing the same kernel exploit communal resources – storage mappings, port mappings, or covert channels – to access each other's data. By monitoring global usage memory or file metadata, attackers can compromise co-located containers, amplifying the risk of unauthorized data exposure. These vulnerabilities can be attributed to vulnerabilities V2, V3 and V4, resulting in the unauthorized leakage of information (TC-1) and potential usurpation (TC-4).

Building on these confidentiality concerns, **elevation of privilege** threats further jeopardize containerized systems by enabling attackers to gain unauthorized control, often leading to full host takeover (Lin et al., 2018). Running containers as root, a default setting in platforms like Docker, allows attackers to execute remote commands or access host files, exploiting the shared kernel architecture. Misconfigured networks, such as Docker's default bridge, exacerbate this risk by permitting unintended inter-container communication, while unnecessary system calls provide additional avenues for exploitation. These privilege escalation paths stem from inherent flaws and configuration oversights in the Linux system (vulnerabilities V1, V3 and V4), violating the principle of secure authorization and leading to the TC-4 consequence.

Based on the analysis presented, we adopt a defensive approach centered on three kernel-level strategies to bolster container security.

First, capability management restricts excessive permissions assigned to container instances, mitigating risks stemming from insecure configurations (Lee et al., 2019). Second, file access control prevents unauthorized access to sensitive host files, countering vulnerabilities such as synergistic power attacks (Gao et al., 2018). Third, monitoring specific kernel function calls, such as bpf_prog, is employed to thwart container escapes and cross-container attacks (He et al., 2023) that exploit powerful kernel extension subsystems. Implemented at the kernel level, these measures deliver direct and robust oversight of container instances, ensuring effective security enforcement.

Notably, our research deliberately narrows its scope by excluding non-direct container attacks, such as resource exhaustion, hardware side-channel attacks, and kernel vulnerability exploits. These attack vectors, while significant, are omitted because their success could destabilize defensive measures or corrupt audit logs, ultimately threatening system availability. Instead, we direct our efforts toward designing and implementing interception mechanisms from a kernel-level perspective, leveraging three core mitigation strategies: capability management, file access control, and targeted monitoring of kernel function calls. Our work further emphasizes the formulation of security policies tailored to these strategies, rather than delving into the creation of minimal access control policies. This focused approach not only clarifies the specific boundaries of our study but also lays the groundwork for future investigations into complementary security challenges.

5. Design of cordon

Designing effective permission controls for containerized environments necessitates reconciling the inherent demands for operational flexibility and dynamic adaptation with the imperative to robustly intercept security-critical events. The core challenge lies in achieving precise, kernel-level event interception through adaptable policy mechanisms. To address this challenge, this section presents a novel permission control method based on the Linux Security Module (LSM) framework. Our approach is centered on two key design principles: (1) the automated generation of tailored permission control policies for protected containers, utilizing a custom security policy syntax proposed in this work, and (2) the dynamic enforcement of these policies by leveraging the event interception capabilities inherent in LSM combined with the runtime programmability of extended Berkeley Packet Filter (eBPF). This integrated strategy enables granular security control and unified policy management for designated containers. The following subsections will elaborate on the design objectives underpinning this method and detail its specific architectural implementation, thereby providing a clear exposition of its theoretical foundations and practical realization path.

5.1. Design goals

Effectively securing containerized environments demands security mechanisms that operate seamlessly within their inherently dynamic and flexible nature, rather than imposing rigid constraints that undermine core operational benefits. To meet this critical requirement, our proposed method, Cordon, establishes a security perimeter by strategically integrating extended Berkeley Packet Filter (eBPF) for granular event visibility and control with the Linux Security Module (LSM) framework for robust policy enforcement. The paramount objective of Cordon is to autonomously mitigate security threats targeting containers – such as unauthorized file access, privilege escalation, and malicious system call sequences – by enforcing precise, kernel-level controls over these critical aspects, all while preserving the essential dynamism and flexibility intrinsic to containerized computation. Therefore, Cordon's design is driven by the following core goals, ensuring its efficacy and practicality within real-world environments:

- **Objective 1: Autonomous Control Capability.** Cordon is designed to provide containers with the ability to autonomously control their own security policies within the kernel. This involves fine-grained management of system resources such as file access, system calls, and permissions, tailored to the specific needs of each container instance. By enabling containers to enforce their own security measures independently, Cordon addresses the need for individualized security controls in diverse and dynamic container environments, enhancing both security and operational efficiency.
- **Objective 2: Simple and Flexible Policy Configuration.** Cordon aims to provide a straightforward and adaptable policy configuration for secure access control. It enables the definition and enforcement of security policies across different resource dimensions, such as processes, instances, containers, and nodes. This allows for consistent security control within the same deployment, minimizing the configuration complexity for users. By supporting dynamic scaling and diverse resource models, Cordon ensures that security policies can be easily tailored to the evolving requirements of containerized applications, thereby maintaining both security and operational flexibility.
- **Objective 3: Ease of Adoption.** Cordon's third objective centers on ensuring the practical adoptability of our solution within operational contexts. Containerized computing environments impose stringent stability requirements on both the kernel and applications. This necessitates that our solution neither alters the kernel nor compromises application performance while implementing secure access control. Cordon relies on the underlying properties of its eBPF implementation to achieve its adoptability guarantees.

5.2. Overview

As shown in Fig. 3, Cordon's architecture strategically integrates two distinct yet interconnected phases to deliver effective container security: Security Policy Generation and Runtime Security Protection. During the Policy Generation phase, Cordon automatically synthesizes customized security policies tailored to individual containers. This is achieved by parsing the container's creation commands or deployment manifest files and augmenting them with predefined default configurations. The resulting policies precisely define permissible actions and access rules for the protected container instance. Subsequently, in the Runtime Security Protection phase, these active policies are enforced within the kernel. Cordon leverages extended Berkeley Packet Filter (eBPF) programs dynamically attached to Linux Security Module (LSM) hooks. This combination enables real-time monitoring and granular interception of security-critical events – such as file access, privilege changes, and system calls – initiated by processes within the target container. Furthermore, detailed audit logs generated during enforcement provide security analysts with actionable insights for continuous policy refinement, strengthening the overall defensive posture. This two-phase design establishes a closed feedback loop, tightly coupling declarative policy specification with precise, kernel-level enforcement to ensure both timely protection and manageable security operations.

The core enforcement mechanism orchestrates a streamlined control flow traversing user and kernel space. This process is initiated after the user inputs the relevant information of the container instance to be monitored. Cordon dynamically generates and activates the corresponding security policy within the kernel. When the application within the monitored container instance issues a system call (e.g., to access a file), the request transitions from user space into the kernel. Here, the kernel performs standard operations: locating the target resource's inode and conducting preliminary Discretionary Access Control (DAC) checks. Crucially, Cordon's eBPF programs, attached to strategically placed LSM hooks, intercept the operation after the initial DAC validation but before the action is finalized. Based on the active container-specific policy, Cordon dynamically influences

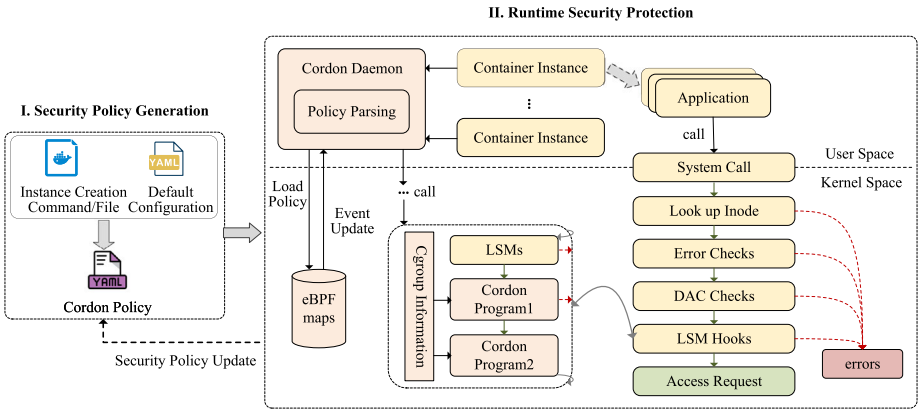


Fig. 3. Cordon overview. Cordon comprises security control policies, a daemon, and self-programming eBPF programs. The system enforces security protection for the relevant containers based on the specified policies.

```
objectives:
  target_layer: container
  target_info:
    name:
      -test
files:
  mode: block
  allow:
    - /
  deny:
    - /proc/sysrq-trigger
    - /proc/self/exe
capabilities:
  mode: block
  allow: []
  deny:
    - cap_sys_admin
    - cap_dac_read_search
bpf:
  mode: block
log:
  format: json
```

Fig. 4. An example of compliance with the Cordon security control policy language.

the LSM hook's decision to either grant or deny the requested access. This specific interception point is vital: it allows Cordon to effectively counter prevalent container threats like unauthorized file access (mitigating Tampering and Information Disclosure) or exploitation of flawed permissions, which DAC alone cannot reliably prevent in containerized computing environments. By dynamically leveraging eBPF atop the LSM framework, Cordon directly addresses the inherent limitations of traditional, statically compiled LSM modules in container contexts. It inherits the robust mediation capabilities of LSM while gaining eBPF's essential advantages: dynamic runtime loading, unparalleled flexibility in policy configuration and update, and fine-grained control scoped precisely to individual containers. This synergy satisfies the dual requirements of autonomous security management and adaptable policy enforcement intrinsic to modern, dynamic containerized deployments.

6. Security policy generation

Before exploring the intricacies of our container security control research, this section lays out the syntax and semantics of the proposed security control policy language, establishing a foundation for the design and implementation of our security mechanisms. To ensure policy files are readable, user-friendly, and easily understood, we have crafted the language syntax using YAML, a widely adopted, human-readable data serialization format ideal for configuration files and data exchange. Python-style indentation is employed to clearly delineate nested structures. As shown in Fig. 4, this design addresses the security needs of modern containerized environments, enabling both tailored policies for specific instances and efficient batch management across multiple instances. The `objectives` field operationalizes this duality by defining hierarchical control scopes through its `target_layer` subfield, which specifies enforcement boundaries ranging from single-process monitoring to cluster-wide Kubernetes deployment governance. Precision is achieved via the `target_info` subfield, which enables granular filtering through key–value pairs (e.g., container root process PID or Kubernetes resource name). In the depicted example, the policy restricts enforcement to the container instance labeled "test", demonstrating context-aware security without compromising orchestration scalability.

The generation of security policies within Cordon adheres to a well-defined set of standards derived from our systematic analysis of mitigation strategies for containerized environments (Section 4.3). These standards prioritize two fundamental requirements: enabling precise interception mechanisms and facilitating autonomous, scalable policy management. Consequently, the syntax and semantics of Cordon's security policy language are explicitly designed around configuring controls for three critical kernel-level operation categories: file access, capability management, and system call execution. For file access control, policies allow administrators to specify granular allow/deny lists for files and directories, ensuring strict containment over data interactions. Regarding capability management, policies enforce continuous monitoring of assigned privileges to prevent dangerous over-privileging. Concerning system calls, Cordon introduces programmable monitoring, empowering users to define custom interception logic for specific, high-risk functions; crucially, this includes proactive interception of the bpf_prog system call, directly addressing its recent exploitation in eBPF-powered container escape attacks. This tripartite focus – on files, capabilities, and critical syscalls – embodies the core security posture enforced by Cordon's generated policies.

Algorithm 1 Automated Security Control Policy Generation Algorithm

Input: Container creation command/file *ContainerConfig*, Default configuration file *DefaultPolicyConfig*
Output: Security control policy *Policy*
// Parse the container creation command/file to extract the target container resource type, relevant information, and allowed access lists
TargetLayer, *TargetInfo*, *AllowedFileList*, *AllowedCapabilityList*, *AllowedSyscallList* ← ParseContainerConfig(*ContainerConfig*)
// Load the interception objects from the default configuration file
DenyObjects ← LoadDenyObjects(*DefaultPolicyConfig*)
// Initialize the security control policy structure
Policy ← InitializePolicy()
// Generate the Objectives field, consisting of target_layer and target_info
Policy.Objectives ← GenerateObjectives(*TargetLayer*, *TargetInfo*)
// For file interception objects, retrieve allowed and denied access lists and write to the policy
Policy.Files.Mode ← "block"
Policy.Files.Allow ← *AllowedFileList*
Policy.Files.Deny ← *DenyObjects.Files*
// For capability interception objects, retrieve allowed and denied access lists and write to the policy
Policy.Capabilities.Mode ← "block"
Policy.Capabilities.Allow ← *AllowedCapabilityList*
Policy.Capabilities.Deny ← *DenyObjects.Capabilities*
// Configure system call interception (BPF) with block mode and allow/deny lists
if $bpf \notin$ *AllowedSyscallList* **then**
    Policy.BPF.Mode ← "block";
    Policy.BPF.Deny ← *DenyObjects.BPF*;
// Set logging format to JSON for auditability
Policy.Log.Format ← "json"
return *Policy*

While the policy language's syntax and semantics are well-defined, manually crafting these policies can be labor-intensive in practice. To streamline this process, Cordon employs an automated policy generation algorithm that leverages container creation commands or files alongside default configurations to produce tailored security policies, significantly easing the configuration burden. Detailed in Algorithm 1, this method extracts critical parameters from container creation data – guided by threat analysis-derived defaults – and substitutes them into a policy template to create a customized policy. It should be noted that to ensure compliance with security principles, this policy template is designed in accordance with standardized security benchmarks (Muench and Hartmann, 2025). When new risks emerge, security analysts can update policies dynamically using insights from audit logs, keeping protections aligned with evolving threats. This blend of automation and adaptability simplifies initial policy creation while bolstering Cordon's ability to deliver ongoing security in dynamic environments. Although this work prioritizes flexible, container-level security controls to mitigate known escape attacks, it does not focus on minimal permission strategies—an area where it remains compatible with approaches like SPEAKER (Lei et al., 2017).

7. Runtime security protection

After obtaining the necessary security policies, Cordon proceeds to enforce runtime security protection for containers, a process depicted in phase II of Fig. 3. This section is pivotal to our study, as it dissects the innovative methods through which Cordon realizes autonomous control and scalable batch management. Additionally, it meticulously describes the interception mechanisms employed by Cordon, elucidating where and how it detects and thwarts suspicious events to maintain the integrity and security of the containerized environment.

Container-level runtime security faces the critical challenge of achieving precise, kernel-level monitoring tailored to specific container applications as defined in configuration files. This demands autonomous control over individual containers while supporting scalable and flexible

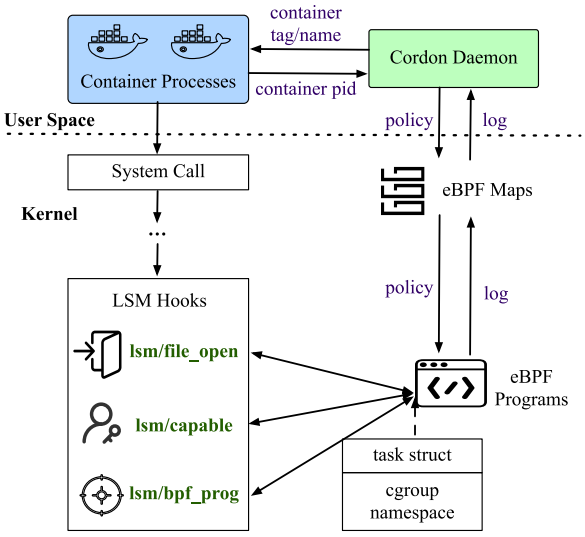


Fig. 5. The detailed security protection mechanism of Cordon, illustrating where and how eBPF programs monitor and control specific container processes.

configurations across multiple resource dimensions, such as processes, instances, and nodes. Unlike prior studies that either focus on single-container access control (Bélair et al., 2021) or uniform host-wide policies (mrtc0 et al., 2022), our approach with Cordon introduces a multi-dimensional security framework. Cordon collects essential information, like process PIDs, from configuration files and container engine APIs, then tracks the cgroup namespaces of targeted processes in a map upon triggering eBPF hooks. At runtime, it matches the current process's cgroup namespace against this map, applying tailored security policies only to matching containers and filtering out others. This ensures accurate, customized protection across diverse container environments.

Effective security hinges on a robust interception mechanism and carefully chosen hook points to counter specific threats. Cordon employs eBPF and LSM, attaching eBPF programs to LSM hooks to gain detailed visibility and control over kernel events. This combination enhances programmability and provides rich security data, enabling precise monitoring and intervention. Addressing key attack vectors identified in Section 4 – file access, permission management, and system function calls – Cordon aligns its defenses with the distinct risks of containerized systems. By focusing on these areas, it delivers targeted protection against both common and emerging threats, adapting to the evolving security needs of container deployments.

As illustrated in Fig. 5, Cordon leverages specific LSM hooks – `file_open`, `capable`, and `bpf_prog` – to regulate file access, permissions, and BPF function calls, respectively. These hooks serve as vital control points, allowing Cordon to oversee and restrict container process behaviors effectively. The file_open hook enforces stringent file access restrictions based on predefined security policies, effectively mitigating the risk of information disclosure arising from weak file system isolation. Building on this, the capable hook limits container permissions, shrinking the attack surface and thwarting potential elevation of privilege exploits. Meanwhile, the bpf_prog hook targets BPF function calls, offering a robust defense against both information disclosure and privilege escalation threats, particularly those exploiting advanced eBPF capabilities. This layered interception strategy not only strengthens Cordon's protective measures but also provides developers with the flexibility to tailor monitoring to specific kernel operations, ensuring adaptability against increasingly complex attacks in containerized environments.

Table 1
Comparison of security access control solutions against container attacks.

Attack category	Attack instances	AppArmor	Bouheki	Cordon
Capabilities abuse	Docker shocker attack that misuses cap_dac_read_search to read arbitrary files on the host.	Enable with complex configurations.	Disable	Enable with automatically generated policies.
Sensitive files	Accessing sensitive files under /proc directory to obtain host-related information.	Enable	Enable	Enable
Offensive eBPF features	Shellcode injection during mprotect system call to escape the container.	Enable with complex configuration.	Disable	Enable with automatically generated policies.

8. Evaluation

Here, we evaluate the effectiveness, generality, and performance of Cordon. We verify Cordon's protective capabilities under its default policy by reproducing three common container attacks, demonstrate Cordon's universality by exhibiting its control over specific container instances across different container engines. Furthermore, we conduct an empirical analysis to evaluate Cordon's performance, showcasing the acceptable overhead in protecting container environments against security threats.

8.1. Experimental setup

For our evaluation, we perform all experiments on a Linux system equipped with kernel version 5.19 and Docker version 21.0. To ensure consistency and reproducibility, we deploy a virtual machine running Ubuntu 22.10, providing a controlled environment for testing. We enable the CONFIG_BPF_LSM kernel compilation option to activate LSM support for eBPF, further configuring the bpf option in the kernel parameters to fully leverage LSM BPF capabilities. This setup enables us to effectively evaluate the performance and security of our container control mechanisms under realistic and well-defined conditions.

8.2. Effectiveness evaluation

To evaluate Cordon's effectiveness in enhancing container security, we adopt a systematic methodology that ensures robust validation of its protective capabilities. The assessment begins by replicating attack processes in an environment where Cordon is disabled, confirming that exploits succeed as expected. Subsequently, we repeat these attacks with Cordon activated, using its automatically generated configurations. This step-by-step approach provides a clear comparison, enabling us to measure Cordon's ability to thwart threats under controlled conditions.

We first examine Cordon's response to capabilities exploitation through the Docker Shocker attack. This scenario allows an attacker to escape a Docker container and access host filesystem contents by leveraging the `open_by_handle_at` system call, which relies on the container's `cap_dac_read_se- arch` capability. Successful execution underscores the critical role of permission management in preventing such breaches. Cordon proves effective here by restricting these capabilities, blocking unauthorized file access and reinforcing container isolation. Next, we assess vulnerabilities tied to pseudo-files in directories like `/proc` and `/sys`, which often leak sensitive information due to inadequate isolation. Cordon counters this risk by enforcing strict access controls, limiting exposure and protecting against data leakage threats. Finally, we explore eBPF-based attacks, where the kernel's eBPF features enable container escapes by exploiting the shared kernel environment. Studies like He et al. (2023) and Fournier et al. (2021) caution against running eBPF applications in containers due to their traceable and programmable nature. Cordon addresses this uniquely by blocking the `bpf_prog` function, halting kernel manipulations from containerized eBPF programs and preventing escapes at their inception.

Table 2
Comparative evaluation of generality features in security access control solutions.

	Container engine			Batch containers	Programmability
	Docker	Podman	kubernetes		
AppArmor	✓	✓	✓	–	✗
Bouheki	–	–	–	✗	–
Cordon	✓	✓	✓	✓	✓

The protective outcomes of Cordon stand out when compared to existing solutions, as summarized in Table 1. Traditional LSM-based tools like AppArmor demand intricate configurations, posing challenges for users, while eBPF-based approaches like Bouheki fall short in managing permissions effectively. In contrast, Cordon integrates eBPF programs with LSM hooks to enforce precise permission controls, streamlined further by its YAML-based policy auto-generation, which enhances usability. Unlike many container security frameworks that neglect eBPF threats – or worse, introduce risks by running as sidecar containers – Cordon operates solely within a trusted host kernel environment. This design minimizes the attack surface and delivers a proactive, comprehensive defense, setting it apart in the landscape of container security solutions.

8.3. Generality evaluation

To demonstrate the generality of our solution, we compare Cordon's application across three dimensions: target container types, target container quantities, and programmability, as shown in Table 2. The category of target container refers to the applicability of the security solution to various container engines. The target container quantities indicate the capability to process multiple containers of the same type in bulk. Programmability assesses whether the solution can be extended and customized for specific security scenarios. In addition, the "–" symbol in the table denotes partial fulfillment of functionality. For instance, Bouheki supports categories of containers such as Docker and Podman but cannot implement access control for specific Docker or Podman containers. It can only manage all containers on the host system. Our comparison of Cordon with solutions such as AppArmor and Bouheki reveals that Cordon excels in cross-container engine compatibility and possesses the ability to handle multiple containers, showcasing ease of expansion and usability.

In terms of container engine compatibility, most solutions are applicable across a variety of container engines. However, some solutions like Bouheki lack the ability to recognize specific container applications, missing out on autonomous security control at the individual container level. In addition, LSM frameworks such as AppArmor face the challenge of complex configuration processes, which can lead to errors during applications. In contrast, Cordon acquires information on target containers through the API interfaces of container engines and retrieves cgroup namespace data of processes within these target containers. By comparing cgroup namespaces, Cordon can adeptly identify processes within specific containers. This method can easily adapt to

Table 3
Performance overhead at different instrumentation points of cordon in microbenchmark tests.

Operation	*file_open*	*capable*	*bpf_prog*
Cordon (ns)	13 446	46 362	45 149

various container engines as long as these engines provide API access to Cordon, and it ingeniously enables autonomous security control for specific containers with simple policy configuration.

Concerning the batch security control of containers, most solutions overlook such needs. However, in containerized computing environments like microservices or serverless architectures, containers with the same functionality are often run in batches for load balancing and stable operation. Solutions like AppArmor and Bouheki neglect security management for batches of similar container applications in these scenarios. Cordon addresses this issue by rapidly identifying these target containers for monitoring and control using tags, pids, and other identifying information.

Furthermore, the expandability of solutions is crucial in adapting to different scenarios and the evolving landscape of cybersecurity threats and defenses. Built on the eBPF framework, Cordon can be customized and modified according to current needs. With a focus on recent attacks leveraging offensive eBPF features to facilitate container escape, we provide control over the `bpf_prog` function based on an extendable and programmable interface. Developers can swiftly develop new interception points and functionalities using this interface, demonstrating Cordon's capability for rapid adaptation and customizability to address emerging security challenges.

8.4. Performance evaluation

Our performance evaluation begins with microbenchmark tests, utilizing the `bpf_ktime_get_ns` helper function to measure the performance overhead at each instrumentation point. As shown in Table 3, Cordon's overhead at each point does not exceed 0.05 ms, most of which comes from the determination of whether the current process belongs to the target container. Next, we perform macrobenchmark tests to assess Cordon's performance overhead for container instances hosting three different types of applications. These applications continuously trigger file open functions, capability checks, and BPF program execution functions during runtime. These container applications with varied focuses enable us to effectively and reasonably test the Cordon's overhead. We use `pidstat` to record the CPU usage during the runtime of Cordon for these container applications, and the CPU utilization remains below 4% for all three types. To further strengthen our evaluation, we also employ the well-known sysbench benchmark to assess the impact of Cordon on CPU, memory, storage, and network performance. As shown in Table 4, we compare the performance of systems with and without Cordon operational. The data presented is the average of ten recordings.

9. Discussion

This research centers on designing and implementing a flexible security control mechanism at the container level, with a focus on automated policy generation tailored to container management. The developed approach effectively addresses three prevalent escape attack vectors, establishing a solid base for enhancing container security. Looking ahead, we aim to advance this work by investigating the generation of minimal security control policies specific to containers. This evolution seeks to alleviate the workload of security personnel by reducing manual analysis, thereby optimizing the configuration process while upholding robust protection. Such progress in automating least-privilege policy identification is expected to boost both the efficiency and adaptability of container security in ever-changing environments.

10. Related work

Linux Security Modules (LSMs) form a lightweight, versatile access control framework embedded in the Linux kernel, enabling on-demand loading of security modules such as SELinux (Wikipedia, 2025b), AppArmor (Wikipedia, 2025a), and Tomoyo (NTT DATA Corporation, 2025). These modules enforce Mandatory Access Control (MAC) at the kernel level through policy files that specify permissible process-resource interactions. By restricting container processes from accessing the host or other containers' resources, LSMs significantly bolster container security, mitigating risks like container escapes and cross-container attacks.

In container security research, SELinux and AppArmor stand out as the most prevalent LSMs. Bacis et al. (2015) proposed a method to bind SELinux policies to Docker container images, allowing maintainers to distribute tailored policy modules alongside images. This approach empowers users – even those unfamiliar with SELinux syntax – to run containers with specific policies, addressing the problem of over-privileged containers sharing uniform SELinux configurations. Similarly, Mattetti et al. (2015) introduced LiCShield, a framework that analyzes container images to generate rule sets capturing container behavior, automatically producing AppArmor profiles to ensure secure runtime execution. However, Loukidis-Andreou et al. (2018b) critiqued LiCShield's image-based approach as inadequate against cross-container attacks, proposing Docker-sec instead. Docker-sec integrates static analysis of image creation parameters with dynamic runtime monitoring to create comprehensive container profiles, applying AppArmor policies to all critical Docker components for enhanced protection. Building on these efforts, Zhu and Gehrmann (2021b) developed Lic-Sec, an AppArmor profile generator that merges LiCShield and Docker-sec's strengths, validated through real-world attack scenarios.

Despite their strengths, LSMs face limitations. MP et al. (2016) noted that while SELinux and AppArmor mitigate container risks, their policies are often shared across all containers, limiting per-container customization. To address this, Sun et al. (2018) introduced Security Namespaces to enhance resource isolation. Additionally, Findlay et al. (2021) proposed BPFCONTAIN, leveraging eBPF to attach programs to LSM hooks and kernel functions, enabling fine-grained, container-specific policy enforcement in kernel space.

In summary, LSMs provide robust container security through kernel-level MAC, with SELinux and AppArmor offering flexible policy-based resource control. Nevertheless, challenges such as policy sharing and isolation gaps persist, driving innovations like eBPF to refine LSM applications in container security.

11. Conclusion

In this paper, we present Cordon, a fine-grained access control system for enhanced security in containerized environments. Addressing the need for autonomous security control and flexible policy configuration in containerized computing environments, Cordon proposes an automated policy generation algorithm and designs a runtime security protection mechanism. Specifically, Cordon generates control policies by extracting information from container deployment files, subsequently utilizing eBPF programs mounted on LSM hooks to monitor and intercept related activities of targeted containers according to these policies. We demonstrate the effectiveness and generality of our solution by reproducing various attacks across different container engines. The evaluation results show that our solution is more effective than existing access control methods in containerized computing environments and introduces an acceptable additional overhead.

CRediT authorship contribution statement

Qiqing Deng: Writing – original draft, Software, Methodology, Investigation. **Zhen Xu:** Writing – review & editing. **Qihui Zhou:** Writing – review & editing. **Yan Zhang:** Funding acquisition.

Table 4
Performance evaluation.

	CPU (events per second)	Memory (Operations per second)	Storage (read Throughput)
without Cordon	425.240	17504.1 MiB/sec	18.66 MiB/s
with Cordon	435.155	18634.4 MiB/sec	14.94 MiB/s
incremental overhead	2.33%	6.46%	19.93%

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (No. 2023YFB3106302).

Data availability

No data was used for the research described in the article.

References

Ahmad, I., AlFailakawi, M.G., AlMutawa, A., Alsalman, L., 2022. Container scheduling techniques: A survey and assessment. J. King Saud Univ.-Comput. Inf. Sci. 34 (7), 3934–3947.

Aruna, K., Pradeep, G., 2020. Performance and scalability improvement using IoT-based edge computing container technologies. SN Comput. Sci. 1 (2), 91.

Bacis, E., Mutti, S., Capelli, S., Paraboschi, S., 2015. DockerPolicyModules: mandatory access control for docker containers. In: 2015 IEEE Conference on Communications and Network Security. CNS, IEEE, pp. 749–750.

Bélair, M., Laniepce, S., Menaud, J.-M., 2021. SNAPPY: programmable kernel-level policies for containers. In: Proceedings of the 36th Annual ACM Symposium on Applied Computing. SAC '21, Association for Computing Machinery, New York, NY, USA, pp. 1636–1645. http://dx.doi.org/10.1145/3412841.3442037.

Brimhall, B., Garrard, J., De La Garza, C., Coffman, J., 2023. A comparative analysis of linux mandatory access control policy enforcement mechanisms. In: Proceedings of the 16th European Workshop on System Security. EUROSEC '23, Association for Computing Machinery, New York, NY, USA, pp. 1–7. http://dx.doi.org/10.1145/3578357.3589454.

Findlay, W., Barrera, D., Somayaji, A., 2021. Bpfcontain: Fixing the soft underbelly of container security. arXiv preprint arXiv:2102.06972.

Findlay, W., Somayaji, A., Barrera, D., 2020. bpfbox: Simple precise process confinement with eBPF. In: Proceedings of the 2020 ACM SIGSAC Conference on Cloud Computing Security Workshop. CCSW '20, Association for Computing Machinery, New York, NY, USA, pp. 91–103. http://dx.doi.org/10.1145/3411495.3421358.

Flauzac, O., Mauhourat, F., Nolot, F., 2020. A review of native container security for running applications. Procedia Comput. Sci. 175, 157–164.

Fournier, G., Baubeau, S., Baubeau, S., 2021. eBPF, I thought we were friends.

Gao, X., Steenkamer, B., Gu, Z., Kayaalp, M., Pendarakis, D., Wang, H., 2018. A study on the security implications of information leakages in container clouds. IEEE Trans. Dependable Secur. Comput. 18 (1), 174–191.

He, Y., Guo, R., Xing, Y., Che, X., Sun, K., Liu, Z., Xu, K., Li, Q., 2023. Cross container attacks: The bewildered {$eBPF$} on clouds. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 5971–5988.

Kaur, K., Dhand, T., Kumar, N., Zeadally, S., 2017. Container-as-a-service at the edge: Trade-off between energy efficiency and service availability at fog nano data centers. IEEE Wirel. Commun. 24 (3), 48–56. http://dx.doi.org/10.1109/MWC.2017.1600427.

Lee, S., Seo, J., Nam, J., Shin, S., 2019. Poster: Tclp: Enforcing least privileges to prevent containers from kernel vulnerabilities. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 2665–2667.

Lei, L., Sun, J., Sun, K., Shenefiel, C., Ma, R., Wang, Y., Li, Q., 2017. Speaker: Split-phase execution of application containers. In: Detection of Intrusions and Malware, and Vulnerability Assessment: 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings 14. Springer, pp. 230–251.

Li, Z., Liu, W., Wang, X., Yuan, B., Tian, H., Jin, H., Yan, S., 2023. Lost along the way: Understanding and mitigating path-misresolution threats to container isolation. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 3063–3077.

Lin, X., Lei, L., Wang, Y., Jing, J., Sun, K., Zhou, Q., 2018. A measurement study on linux container security: Attacks and countermeasures. In: Proceedings of the 34th Annual Computer Security Applications Conference. pp. 418–429.

Loukidis-Andreou, F., Giannakopoulos, I., Doka, K., Koziris, N., 2018a. Docker-sec: A fully automated container security enhancement mechanism. In: 2018 IEEE 38th International Conference on Distributed Computing Systems. ICDCS, IEEE, pp. 1561–1564.

Loukidis-Andreou, F., Giannakopoulos, I., Doka, K., Koziris, N., 2018b. Docker-sec: A fully automated container security enhancement mechanism. In: 2018 IEEE 38th International Conference on Distributed Computing Systems. ICDCS, IEEE, pp. 1561–1564.

Mattetti, M., Shulman-Peleg, A., Allouche, Y., Corradi, A., Dolev, S., Foschini, L., 2015. Securing the infrastructure and the workloads of linux containers. In: 2015 IEEE Conference on Communications and Network Security. CNS, IEEE, pp. 559–567.

M.P., A.R., Kumar, A., Pai, S.J., Gopal, A., 2016. Enhancing security of docker using linux hardening techniques. In: 2016 2nd International Conference on Applied and Theoretical Computing and Communication Technology. ICATccT, pp. 94–99. http://dx.doi.org/10.1109/ICATCCT.2016.7911971.

MP, A.R., Kumar, A., Pai, S.J., Gopal, A., 2016. Enhancing security of docker using linux hardening techniques. In: 2016 2nd International Conference on Applied and Theoretical Computing and Communication Technology. ICATccT, IEEE, pp. 94–99.

mrtc0, et al., 2022. bouheki: Krsi(eBPF+lsm) based linux security auditing tool. https://github.com/mrtc0/bouheki. (Accessed 24 April 2025).

Muench, P., Hartmann, C., 2025. CIS docker benchmark. https://github.com/dev-sec/cis-docker-benchmark.

NTT DATA Corporation, 2025. Tomoyo. https://tomoyo.sourceforge.net/about.html.en. (Accessed 24 April 2025).

Pérez, A., Moltó, G., Caballer, M., Calatrava, A., 2018. Serverless computing for container-based architectures. Future Gener. Comput. Syst. 83, 50–59. http://dx.doi.org/10.1016/j.future.2018.01.022, URL https://www.sciencedirect.com/science/article/pii/S0167739X17316485.

Rice, L., 2023. Learning eBPF. " O'Reilly Media, Inc.".

sandeepseeram, 2021. Privileged container escape. https://www.sandeepseeram.com/post/docker-privileged-container-escape. (Accessed 24 April 2025).

Schreuders, Z.C., McGill, T., Payne, C., 2013. The state of the art of application restrictions and sandboxes: A survey of application-oriented access controls and their shortfalls. Comput. Secur. 32, 219–241. http://dx.doi.org/10.1016/j.cose.2012.09.007, URL https://www.sciencedirect.com/science/article/pii/S0167404812001435.

Songi Gwak, S.J., 2023. Container instrumentation and enforcement system for runtime security of kubernetes platform with eBPF. Intell. Autom. Soft Comput. 37 (2), 1773–1786. http://dx.doi.org/10.32604/iasc.2023.039565, URL http://www.techscience.com/iasc/v37n2/53245.

Souppaya, M., Morello, J., Scarfone, K., 2017. Application Container Security Guide. Technical Report, National Institute of Standards and Technology.

Sultan, S., Ahmad, I., Dimitriou, T., 2019. Container security: Issues, challenges, and the road ahead. IEEE Access 7, 52976–52996.

Sun, Y., Safford, D., Zohar, M., Pendarakis, D., Gu, Z., Jaeger, T., 2018. Security namespace: making linux security frameworks available to containers. In: 27th {$USENIX$} Security Symposium. {$USENIX$} Security 18, pp. 1423–1439.

Tak, B., Isci, C., Duri, S., Bila, N., Nadgowda, S., Doran, J., 2017. Understanding security implications of using containers in the cloud. In: 2017 USENIX Annual Technical Conference. USENIX ATC 17, USENIX Association, Santa Clara, CA, pp. 313–319, URL https://www.usenix.org/conference/atc17/technical-sessions/presentation/tak.

Watada, J., Roy, A., Kadikar, R., Pham, H., Xu, B., 2019. Emerging trends, techniques and open issues of containerization: A review. IEEE Access 7, 152443–152472.

Wikipedia, 2025a. AppArmor. https://wiki.ubuntu.com/AppArmor. (Accessed 24 April 2025).

Wikipedia, 2025b. SELinux. https://en.wikipedia.org/wiki/Security-Enhanced_Linux. (Accessed 24 April 2025).

Wong, A.Y., Chekole, E.G., Ochoa, M., Zhou, J., 2023. On the security of containers: Threat modeling, attack analysis, and mitigation strategies. Comput. Secur. 128, 103140.

Wright, C., Cowan, C., Smalley, S., Morris, J., Kroah-Hartman, G., 2002. Linux security modules: General security support for the linux kernel. In: 11th USENIX Security Symposium. USENIX Security 02.

Yang, Y., Shen, W., Ruan, B., Liu, W., Ren, K., 2021. Security challenges in the container cloud. In: 2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications. TPS-ISA, IEEE, pp. 137–145.

Zhu, H., Gehrmann, C., 2021a. Lic-Sec: An enhanced AppArmor docker security profile generator. J. Inf. Secur. Appl. 61, 102924. http://dx.doi.org/10.1016/j.jisa.2021.102924, URL https://www.sciencedirect.com/science/article/pii/S2214212621001435.

Zhu, H., Gehrmann, C., 2021b. Lic-Sec: an enhanced AppArmor docker security profile generator. J. Inf. Secur. Appl. 61, 102924.